■ T-SQL Interview Cheat Sheet

Your compiled learning from practice sessions!

1■■ UNPIVOTING DATA WITH UNION ALL

Pattern: Convert multiple columns into rows

```
WITH UnpivotedData AS (
    SELECT ID, 1 AS LineNumber, Column1 AS Value
    FROM Table WHERE Column1 IS NOT NULL
    UNION ALL
    SELECT ID, 2 AS LineNumber, Column2 AS Value
    FROM Table WHERE Column2 IS NOT NULL
    UNION ALL
    SELECT ID, 3 AS LineNumber, Column3 AS Value
    FROM Table WHERE Column3 IS NOT NULL
)
SELECT * FROM UnpivotedData
WHERE ID IS NOT NULL ORDER BY ID, LineNumber;
```

■■ Common Mistakes

Mistake	Fix
Duplicating UNION ALL blocks	Review each block carefully
Wrong LineNumber values	Use sequential 1, 2, 3, 4...
Missing NULL filters	Add WHERE ColumnX IS NOT NULL

2■■ AND vs OR - DE MORGAN'S LAW ■

The Golden Rule: "Exclude with OR" → "Include with AND"

De Morgan's Law:

```
NOT (A OR B)  =  (NOT A) AND (NOT B)
NOT (A AND B) =  (NOT A) OR (NOT B)
```

Practical Example: Exclude rows where value is NULL OR empty

Approach	SQL	Result
■ WRONG	WHERE col IS NOT NULL OR col <> ''	Includes empty strings!
■ CORRECT	WHERE col IS NOT NULL AND col <> ''	Excludes both

Truth Table:

Value	IS NOT NULL	<> ''	OR	AND

'ABC'	■ TRUE	■ TRUE	■ KEEP	■ KEEP
NULL	■ FALSE	■ FALSE	■	■ EXCLUDE
'' (empty)	■ TRUE	■ FALSE	■ OOPS!	■ EXCLUDE

3■■ FINDING DUPLICATES & MAX VALUES

Approach 1: GROUP BY + Correlated Subquery

```
SELECT CustomerID, CustomerName,
    COUNT(*) AS TotalTransactions,
    MAX(Amount) AS MaxAmount,
    (SELECT TOP(1) TransactionDate FROM Sales s2
     WHERE s2.CustomerID = s.CustomerID
     ORDER BY Amount DESC) AS MaxTransactionDate
FROM Sales s
GROUP BY CustomerID, CustomerName
HAVING COUNT(*) > 1;
```

Approach 2: CTE + ROW_NUMBER (More Performant)

```
WITH RankedData AS (
    SELECT CustomerID, CustomerName, Amount, TransactionDate,
        ROW_NUMBER() OVER (PARTITION BY CustomerID
                           ORDER BY Amount DESC) AS rn,
        COUNT(*) OVER (PARTITION BY CustomerID) AS TotalCount,
        MAX(Amount) OVER (PARTITION BY CustomerID) AS MaxAmount
    FROM Sales
)
SELECT CustomerID, CustomerName, TotalCount, MaxAmount,
       TransactionDate AS MaxTransactionDate
FROM RankedData WHERE rn = 1 AND TotalCount > 1;
```

Key Window Functions:

Function	Purpose
`COUNT(*) OVER (PARTITION BY col)`	Count per group
`MAX(col) OVER (PARTITION BY col)`	Max per group
`ROW_NUMBER() OVER (PARTITION BY col ORDER BY col2)`	Rank within group

4■■ QUERY OPTIMIZATION

Step-by-Step: 1) Analyze execution plan → 2) Clear cache → 3) Identify bottlenecks → 4) Optimize query → 5) Add indexes

Cache Commands (MSSQL):

```
DBCC DROPCLEANBUFFERS;  -- Clears data cache
DBCC FREEPROCCACHE;     -- Clears execution plan cache
```

Common Performance Killers ■

1. Implicit Conversions:

```
-- ■ BAD: Function on column prevents index use
WHERE CAST(CustomerID AS VARCHAR) = '12345'
-- ■ GOOD: Direct comparison
WHERE CustomerID = 12345
```

2. Functions on Columns:

```
-- ■ BAD: Function wraps column
WHERE YEAR(OrderDate) = 2024
-- ■ GOOD: Range comparison
WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'
```

Covering Index (Eliminates Key Lookups):

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID
ON Orders (CustomerID, OrderDate)
INCLUDE (TotalAmount);  -- Include columns needed in SELECT
```

Execution Plan Warning Signs ■■

Operator	Problem
Table Scan	Missing index or can't use existing
Key Lookup	Index doesn't cover all columns
Sort	Consider adding ORDER BY columns to index
Hash Match	Might need better indexes
Fat arrows	Large data movement

5■■ RECURSIVE CTEs (Hierarchical Data)

Use Case: Employee Hierarchy, Org Charts, Bill of Materials

The Pattern: Anchor + Recursive

```
WITH RecursiveCTE AS (
    -- ANCHOR (Base Case): Starting point
    SELECT EmployeeID, Name, ManagerID, 0 AS Level
    FROM Employees
    WHERE ManagerID IS NULL  -- Root node(s)
```

```
    UNION ALL

    -- RECURSIVE: Join back to the CTE itself
    SELECT e.EmployeeID, e.Name, e.ManagerID, r.Level + 1
    FROM Employees e
    JOIN RecursiveCTE r ON e.ManagerID = r.EmployeeID
)
SELECT * FROM RecursiveCTE ORDER BY Level, EmployeeID;
```

Key Components:

Part	Purpose	Example
Anchor	Starting point (no recursion)	WHERE ManagerID IS NULL
UNION ALL	Combines anchor + recursive	Required keyword
Recursive Member	References the CTE itself	JOIN RecursiveCTE r ON...
Level Tracking	Track depth in hierarchy	r.Level + 1

Building a Path:

```
-- Add to recursive: Build path string
CAST(eh.Path + ' > ' + e.Name AS VARCHAR(500)) AS Path
-- Output: Alice > Bob > David > Grace
```

Safety - Max Recursion:

```
OPTION (MAXRECURSION 200);  -- Increase limit
OPTION (MAXRECURSION 0);    -- Unlimited (careful!)
```

6■■ QUICK SYNTAX REFERENCE

Aggregate + GROUP BY:

```
SELECT col, COUNT(*), MAX(val), SUM(val)
FROM table GROUP BY col HAVING COUNT(*) > 1;
```

Window Functions:

```
SELECT col,
    ROW_NUMBER() OVER (PARTITION BY group_col ORDER BY sort_col) AS rn,
    COUNT(*) OVER (PARTITION BY group_col) AS group_count,
    MAX(val) OVER (PARTITION BY group_col) AS group_max
FROM table;
```

CTE Pattern:

```
WITH CTE_Name AS (
    SELECT ... FROM ...
)
SELECT * FROM CTE_Name WHERE ...;
```

Correlated Subquery:

```
SELECT outer.col,
    (SELECT TOP 1 inner.val FROM table inner
     WHERE inner.key = outer.key
     ORDER BY inner.sort_col DESC) AS subquery_result
FROM table outer;
```

■ INTERVIEW READY CHECKLIST

- ■ GROUP BY with aggregate functions
- ■ HAVING clause for filtering groups
- ■ Correlated subqueries with TOP 1
- ■ Window functions (ROW_NUMBER, COUNT OVER, MAX OVER)
- ■ PARTITION BY for grouping within windows
- ■ CTEs for complex queries
- ■ Recursive CTEs for hierarchical data
- ■ UNION ALL for unpivoting
- ■ De Morgan's Law (AND vs OR)
- ■ Index optimization
- ■ Execution plan analysis

Good luck with your interviews! ■